

LangChain-Core

源码架构分析文档

基于 v0.1.16 版本源码深度分析

涵盖核心类设计、流程图、时序图等架构说明

langchain-core 是 LangChain 生态的基础抽象层
定义了 Runnable、消息、模型、工具等核心协议

目录

一、项目整体架构概览

- 1.1 模块组织结构
- 1.2 分层架构图
- 1.3 核心类全景继承关系图

二、Runnable 编排框架 (LCEL 核心)

- 2.1 Runnable 核心类设计
- 2.2 管道操作符工作流程图
- 2.3 RunnableSequence 串行执行时序图
- 2.4 RunnableParallel 并行执行时序图
- 2.5 特殊 Runnable 实现

三、语言模型抽象层

- 3.1 模型类继承体系
- 3.2 ChatModel.invoke() 完整调用时序图
- 3.3 流式生成(stream)时序图
- 3.4 with_structured_output 流程图

四、消息类型与 Prompt 模板系统

- 4.1 消息类型继承体系
- 4.2 Prompt 模板系统设计
- 4.3 Prompt 到消息的转换流程图

五、回调与追踪系统

- 5.1 回调系统架构图
- 5.2 事件分发时序图
- 5.3 追踪器与回调的关系

六、工具系统与输出解析

- 6.1 工具系统核心设计
- 6.2 工具调用完整时序图
- 6.3 输出解析器体系

七、数据层：文档、检索与向量存储

- 7.1 数据层类图
- 7.2 RAG 完整流程图

八、序列化与反序列化系统

8.1 序列化机制

8.2 安全反序列化流程图

一、项目整体架构概览

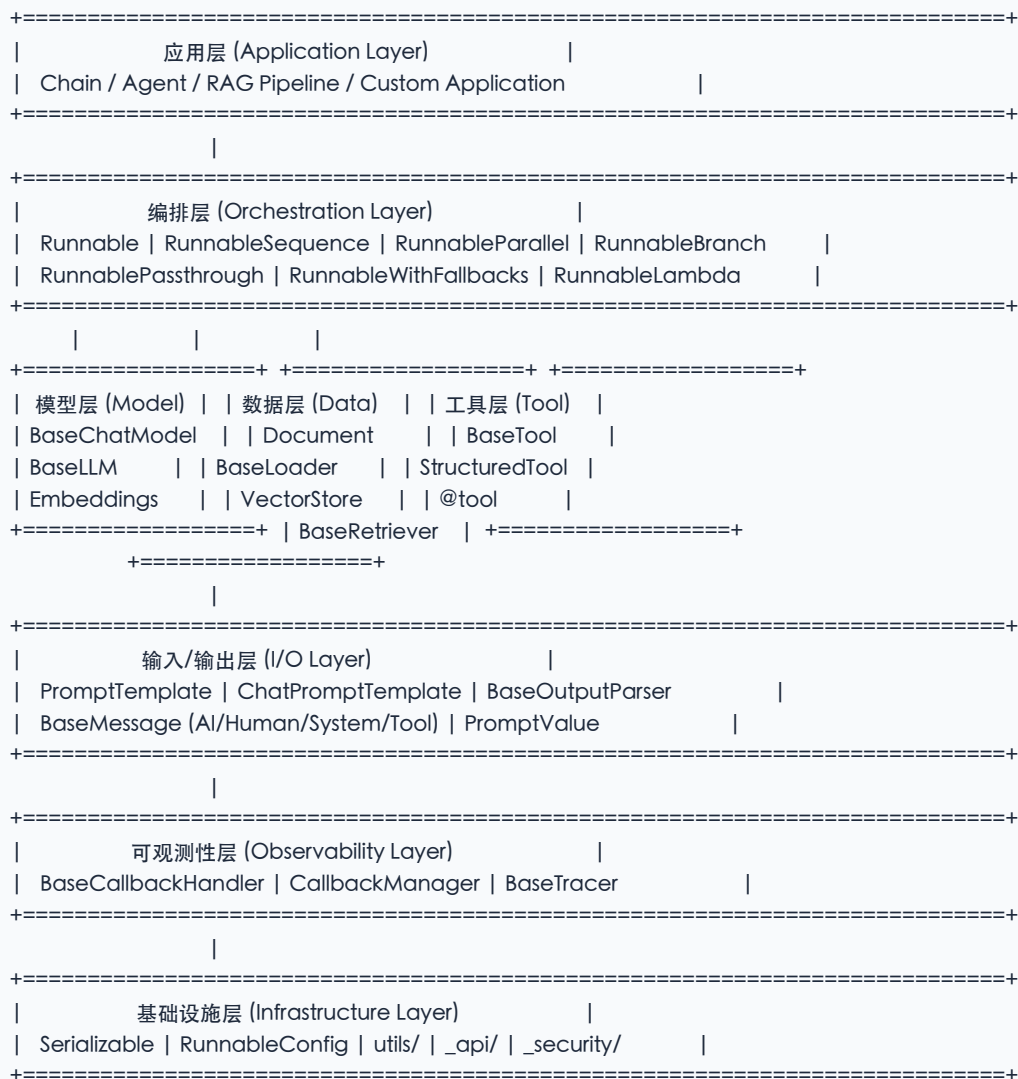
1.1 模块组织结构

langchain-core 是整个 LangChain 生态的基础层，定义了所有核心抽象、协议和基础类型。整个包按功能职责分为 20 个子包，约 195 个 Python 文件。下面按功能分层说明各模块的职责：

功能层	模块	核心职责
核心编排层	runnables/	LCEL 核心，Runnable 基类及管道操作符、序列/并行执行
模型抽象层	language_models/	BaseLLM、BaseChatModel 等语言模型基类
消息系统	messages/	BaseMessage 及 AI/Human/System/Tool 消息类型
输入/输出层	prompts/, output_parsers/, outputs/	Prompt 模板系统和结构化输出解析
数据层	documents/, document_loaders/, vectorstores/, indexing/	文档模型、加载器、向量存储、索引
工具与 Agent	tools/, agents.py	工具抽象、@tool 装饰器、Agent 基础定义
可观测性	callbacks/, tracers/	回调管理器、追踪器、事件分发
基础设施	utils/, load/, _api/, _security/	通用工具、序列化/反序列化、API 标记、安全

1.2 分层架构图

图 1-1: LangChain-Core 分层架构图



1.3 核心类全景继承关系图

图 1-2: 核心类继承全景图

```

BaseModel (Pydantic)
|
+-- Serializable (ABC)      ← 序列化基础能力
|
|   +-- BaseMedia           ← 媒体基类
|   |   +-- Document        ← 文档模型 (page_content + metadata)
|   |   +-- Blob            ← 原始数据 (懒加载)
|   |
|   +-- BaseMessage         ← 消息基类 (content + type)
|   |   +-- HumanMessage    ← 用户消息
|   |   +-- AIMessage       ← AI 回复 (含 tool_calls)
|   |   +-- SystemMessage   ← 系统提示
|   |   +-- ToolMessage     ← 工具返回 (含 tool_call_id)
|   |   +-- BaseMessageChunk ← 流式消息块 (支持 __add__ 合并)
|   |
|   +-- Runnable (ABC, Generic[I, O]) ← 核心编排协议
|   |
|   |   +-- RunnableSerializable ← 可序列化的 Runnable
|   |   |   +-- RunnableSequence ← 串行管道 (A | B | C)
|   |   |   +-- RunnableParallel ← 并行执行 ({k1: R1, k2: R2})
|   |   |   +-- RunnablePassthrough ← 透传 / assign / pick
|   |   |   +-- RunnableBranch   ← 条件分支路由
|   |   |   +-- RunnableBindingBase ← 绑定配置/参数
|   |   |   |   +-- RunnableWithFallbacks ← 故障转移
|   |   |   |   +-- RunnableWithMessageHistory ← 消息历史
|   |   |   +-- BaseLanguageModel ← 语言模型基类
|   |   |   |   +-- BaseChatModel ← 聊天模型 (输出 AIMessage)
|   |   |   |   +-- BaseLLM      ← 文本模型 (输出 str)
|   |   |   +-- BasePromptTemplate ← Prompt 模板基类
|   |   |   +-- BaseTool         ← 工具基类
|   |   |   +-- BaseOutputParser ← 输出解析基类
|   |   |   +-- BaseRetriever    ← 检索器基类
|   |   |
|   |   +-- RunnableLambda      ← 包装普通函数
|   |   +-- RunnableGenerator   ← 包装生成器函数

```

二、Runnable 编排框架（LCEL 核心）

2.1 Runnable 核心类设计

Runnable 是 LangChain Expression Language (LCEL) 的核心协议。它是一个泛型抽象基类 `Runnable[Input, Output]`，定义了统一的执行接口。所有 LangChain 组件（模型、Prompt、解析器、检索器、工具等）都实现了此协议，因此可以通过管道操作符自由组合。

核心设计原则：只有 `invoke()` 是唯一的抽象方法，其它所有方法都有基于它的默认实现。

方法	类型	默认行为	子类可覆写优化
<code>invoke(input)</code>	抽象方法	子类必须实现	-
<code>ainvoke(input)</code>	异步版本	<code>run_in_executor(invoke)</code>	原生 <code>async</code> 实现
<code>batch(inputs)</code>	批量执行	线程池并行调用 <code>invoke</code>	批量 API 调用
<code>abatch(inputs)</code>	异步批量	<code>asyncio.gather(ainvoke...)</code>	原生异步批量
<code>stream(input)</code>	流式输出	<code>yield invoke(input)</code>	真正的逐 token 流式
<code>astream(input)</code>	异步流式	<code>yield await ainvoke(input)</code>	原生异步流式

辅助方法（`with_*` 系列）：

方法	返回类型	功能说明
<code>with_config(config)</code>	<code>RunnableBinding</code>	绑定固定配置
<code>with_retry(stop_after_attempt=3)</code>	<code>RunnableRetry</code>	添加重试策略
<code>with_fallbacks(fallbacks)</code>	<code>RunnableWithFallbacks</code>	添加故障转移方案
<code>with_listeners(on_start, on_end)</code>	<code>RunnableBinding</code>	添加生命周期监听
<code>pipe(*others)</code>	<code>RunnableSequence</code>	等价于 <code>self others[0] ...</code>
<code>pick(keys)</code>	<code>RunnablePick</code>	从 dict 输出中选取 key
<code>assign(**kwargs)</code>	<code>RunnableAssign</code>	向 dict 输出中添加新字段

2.2 管道操作符 (|) 工作流程图

管道操作符是 LCEL 的核心语法糖，通过 Python 的 `__or__` 和 `__ror__` 魔术方法实现。`coerce_to_runnable()` 函数负责将各种类型自动转换为 Runnable：

图 2-1：管道操作符 (|) 类型转换与组合流程

```
用户表达式: prompt | model | output_parser

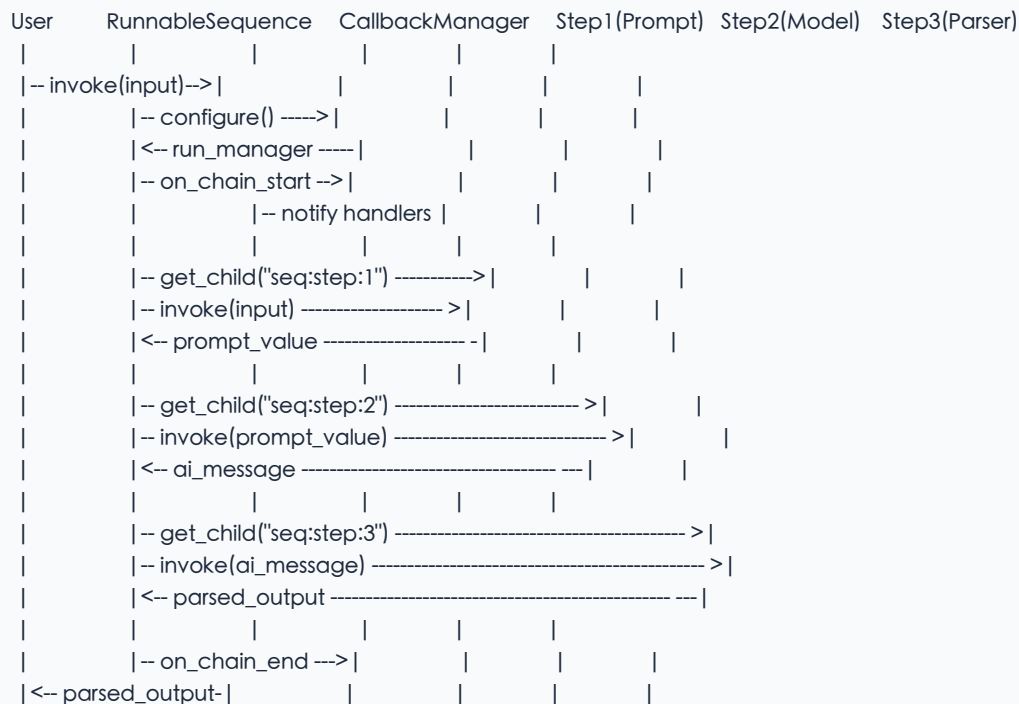
步骤1: prompt.__or__(model)
|
+-- coerce_to_runnable(model)
|   +-- isinstance(model, Runnable) ? --> 直接使用
|   +-- callable(model) ? -----> RunnableLambda(model)
|   +-- isinstance(model, dict) ? ---> RunnableParallel(model)
|   +-- is_generator(model) ? -----> RunnableGenerator(model)
|
+-- 返回 RunnableSequence(prompt, model)

步骤2: sequence.__or__(output_parser)
|
+-- 优化: 如果两边都是 RunnableSequence, 展平所有步骤避免嵌套
|
+-- 返回 RunnableSequence(prompt, model, output_parser)
    first=prompt, middle=[model], last=output_parser

最终结构: RunnableSequence
+-- first: BasePromptTemplate (格式化输入 -> PromptValue)
+-- middle: [BaseChatModel] (生成回复 -> AIMessage)
+-- last: BaseOutputParser (解析输出 -> 结构化结果)
```

2.3 RunnableSequence 串行执行时序图

图 2-2: RunnableSequence.invoke() 执行时序



核心: 前一步输出 = 下一步输入, 每步注册为父 run 的子 run (seq:step:N)

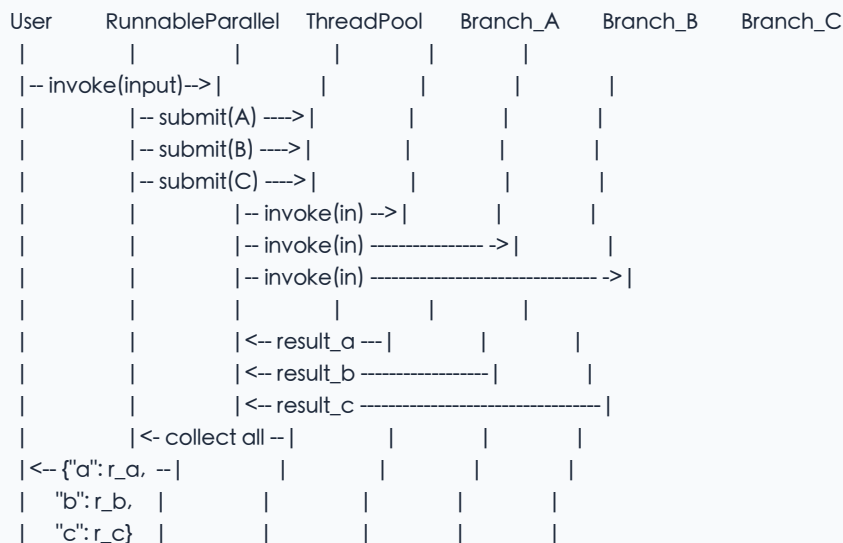
流式执行优化: RunnableSequence 的 stream() 方法通过将每个步骤的 transform() 输出作为一个步骤的输入迭代器, 实现了真正的流式管道。数据不需要等待前一步完全结束, 而是以 chunk 为单位逐步流转。

RunnableSequence.batch() 优化策略:

不是对每个输入单独调用 invoke, 而是逐 step 调用 step.batch(all_inputs)。这让底层 LLM 等组件可以利用自身的批量 API 能力, 大幅提升吞吐量。

2.4 RunnableParallel 并行执行时序图

图 2-3: RunnableParallel.invoke() 并行执行时序



2.5 特殊 Runnable 实现

类	核心逻辑	典型用法
RunnableLambda	包装 callable/coroutine 为 Runnable	runnable (lambda x: x['key'])
RunnablePassthrough	透传输入，可执行副作用函数	RunnablePassthrough()
RunnableAssign	将 mapper 结果合并到输入 dict	RunnablePassthrough.assign(key=runnable)
RunnablePick	从 dict 选取指定 key	runnable.pick('name')
RunnableBranch	条件路由：按序检查条件，执行匹配分支	RunnableBranch((cond1, r1), (cond2, r2), default)
RunnableWithFallbacks	故障转移：主 Runnable 失败时尝试后备	runnable.with_fallbacks([fallback1, fallback2])
RunnableWithMessageHistory	自动加载/保存消息历史到会话存储	runnable.with_message_history(get_history)
RunnableBinding	绑定固定参数或配置到 Runnable	model.bind(temperature=0)

三、语言模型抽象层

3.1 模型类继承体系

图 3-1：语言模型类继承体系

```
RunnableSerializable[LanguageModelInput, LanguageModelOutputVar]
|
+-- BaseLanguageModel[OutputVar] (ABC)    ← 顶层抽象基类
|     字段: cache, verbose, callbacks, tags, metadata
|     方法: get_token_ids(), get_num_tokens()
|
+-- BaseChatModel (BaseLanguageModel[AIMessage]) ← 聊天模型基类
|   | 输入: list[BaseMessage]
|   | 输出: AIMessage
|   | 抽象方法: _generate(messages) -> ChatResult
|   | 可选方法: _stream(), _astream(), bind_tools()
|   | 高级方法: with_structured_output(schema)
|   | 字段: rate_limiter, disable_streaming, profile
|   |
|   +-- SimpleChatModel    ← 便捷子类: 只需实现 _call()
|
+-- BaseLLM (BaseLanguageModel[str]) ← 文本补全模型基类
|   | 输入: list[str]
|   | 输出: str
|   | 抽象方法: _generate(prompts: list[str]) -> LLMResult
|   |
|   +-- LLM    ← 便捷子类: 只需实现 _call(prompt)

类型别名:
LanguageModelInput = PromptValue | str | Sequence[MessageLikeRepresentation]
LanguageModelOutput = BaseMessage | str
```

实现自定义模型的最小接口：

场景	继承类	必须实现	可选实现
聊天模型	BaseChatModel	<code>_generate(messages)->ChatResult</code> <code>+ _llm_type</code>	<code>_stream, _agenerate,</code> <code>bind_tools</code>
简单聊天模型	SimpleChatModel	<code>_call(messages)->str + _llm_type</code>	-
传统 LLM	BaseLLM	<code>_generate(prompts:list[str])->LLMResult</code> <code>+ _llm_type</code>	<code>_stream, _agenerate</code>

简单 LLM	LLM	_call(prompt:str)->str + _llm_type	_acall, _stream
--------	-----	------------------------------------	-----------------

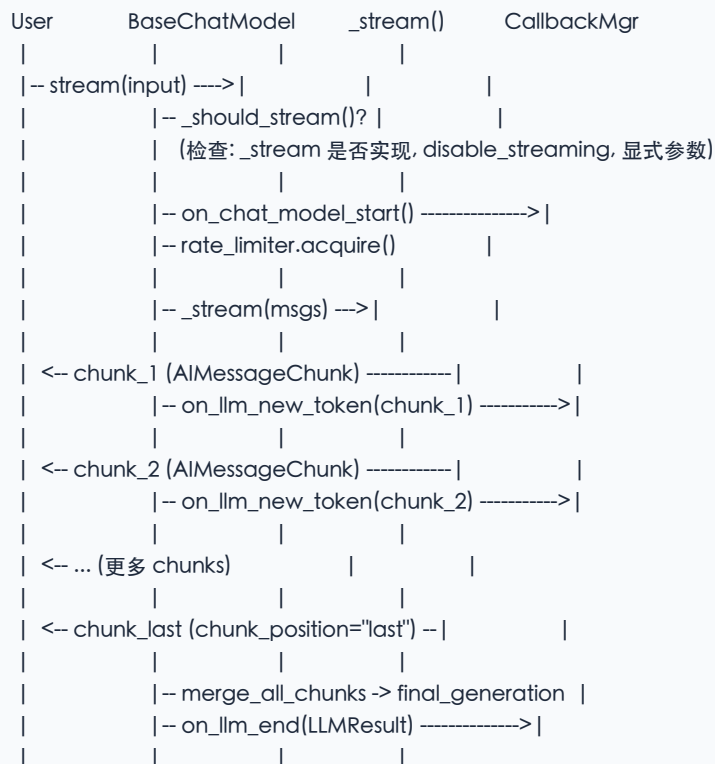
3.2 BaseChatModel.invoke() 完整调用时序图

图 3-2: ChatModel.invoke() 调用链时序图



3.3 流式生成 (stream) 时序图

图 3-3: BaseChatModel.stream() 流式生成时序图



每个 chunk 都是 AIMessageChunk, 通过 __add__ 可以逐步合并为完整 AIMessage
 chunk 属性: content, tool_call_chunks, usage_metadata, chunk_position

3.4 with_structured_output 流程图

图 3-4: with_structured_output() 组装流程

```
model.with_structured_output(schema, include_raw=False)
|
+-- 1. 检查 bind_tools 是否实现 (否则 raise NotImplementedError)
|
+-- 2. llm = self.bind_tools([schema], tool_choice="any")
|    将 schema 绑定为工具，强制模型使用该工具
|
+-- 3. 根据 schema 类型选择解析器:
|    +-- Pydantic BaseModel --> PydanticToolsParser(first_tool_only=True)
|    +-- dict / JSON Schema --> JsonOutputKeyToolsParser(first_tool_only=True)
|
+-- 4. 组装 Runnable 链:
|    |
|    +-- include_raw=False:
|    |    return llm | output_parser
|    |    输入 -> AIMessage(tool_calls) -> Pydantic实例/dict
|    |
|    +-- include_raw=True:
|    |    return RunnableMap(raw=llm) | parser_with_fallback
|    |    输出: {"raw": AIMessage, "parsed": Model | dict, "parsing_error": Exception | None}
```

四、消息类型与 Prompt 模板系统

4.1 消息类型继承体系

图 4-1：消息类型继承体系与核心字段

```
Serializable
|
+-- BaseMessage          ← 消息基类
    字段: content (str | list[str | dict]) ← 支持纯文本和多模态
          additional_kwargs: dict          ← 模型提供商附加数据
          response_metadata: dict          ← 响应元数据
          type: str                        ← 类型判别符
          name, id: str | None             ← 可选标识
    属性: content_blocks -> list[ContentBlock] ← 标准化多模态访问
          text -> str                      ← 纯文本提取
|
+-- HumanMessage (type="human") ← 用户消息 (最简单)
+-- AIMessage (type="ai") ← AI 回复
    |  字段: tool_calls, invalid_tool_calls, usage_metadata
+-- SystemMessage (type="system") ← 系统提示
+-- ToolMessage (type="tool") ← 工具返回结果
    |  字段: tool_call_id (必填), artifact, status
+-- ChatMessage (type="chat") ← 通用消息 (含 role 字段)
|
+-- BaseMessageChunk      ← 流式消息块
    方法: __add__(other) -> 合并两个 chunk
    |
    +-- AIMessageChunk (多重继承: AIMessage + BaseMessageChunk)
        字段: tool_call_chunks, chunk_position
        合并策略: content拼接, tool_call_chunks按index对齐, usage_metadata相加

标准 ContentBlock 类型:
TextContentBlock | ImageContentBlock | ToolCall | ToolCallChunk
ReasoningContentBlock | Citation | NonStandardContentBlock
```

4.2 Prompt 模板系统设计

图 4-2: Prompt 模板类继承体系

```

RunnableSerializable[dict, PromptValue]
|
+-- BasePromptTemplate (ABC)      ← Prompt 模板基类
   字段: input_variables, optional_variables, partial_variables
   方法: invoke(dict) -> PromptValue
   |
   +-- StringPromptTemplate (ABC) ← 字符串模板
   |   | format_prompt() -> StringPromptValue
   |   |
   |   +-- PromptTemplate      ← 纯文本模板
   |       模板格式: f-string(默认) | mustache | jinja2
   |       "Hello {name}, welcome to {place}"
   |
   +-- BaseChatPromptTemplate (ABC) ← 对话模板
       | format_prompt() -> ChatPromptValue
       |
       +-- ChatPromptTemplate ← 对话模板 (核心)
           字段: messages: list[MessageLike]

BaseMessagePromptTemplate (ABC)    ← 单条消息模板
|
+-- MessagesPlaceholder            ← 动态消息列表占位符
|   ("placeholder", "{chat_history}")
|
+-- _StringImageMessagePromptTemplate ← 支持多模态
    +-- HumanMessagePromptTemplate
    +-- AIMessagePromptTemplate
    +-- SystemMessagePromptTemplate

ChatPromptTemplate 支持 5 种输入格式:
1. BaseMessagePromptTemplate 实例
2. BaseMessage 实例
3. ("human", "{input}") 元组
4. (HumanMessage, "{input}") 元组
5. "{input}" 字符串 (默认 human)

```

4.3 Prompt 到消息的完整转换流程图

图 4-3: 从用户输入到模型输入的完整数据流

```

用户输入 dict: {"input": "What is LangChain?", "chat_history": [...]}
|
v
BasePromptTemplate.invoke(input_dict)
|
+-- _validate_input()          验证变量完整性
|
+-- format_prompt(**kwargs)
|
+-- [PromptTemplate 路径]
|   StringPromptTemplate.format_prompt()
|   -> format(**kwargs)      模板字符串替换
|   -> StringPromptValue(text=...)
|       .to_messages() -> [HumanMessage(content=text)]
|
+-- [ChatPromptTemplate 路径]
    BaseChatPromptTemplate.format_prompt()
    -> format_messages(**kwargs)
        遍历 self.messages:
        +-- BaseMessage 实例 -> 直接添加 (静态消息)
        +-- MessagePromptTemplate -> .format_messages()
            |   +-- 纯文本: format() -> Message(content=str)
            |   +-- 多模态: 遍历子模板 -> Message(content=[dict...])
        +-- MessagesPlaceholder -> 注入外部消息列表
        -> ChatPromptValue(messages=[SystemMessage, HumanMessage, ...])
|
v
PromptValue
+-- .to_messages() -> list[BaseMessage] 供 ChatModel 使用
+-- .to_string() -> str                供 LLM 使用

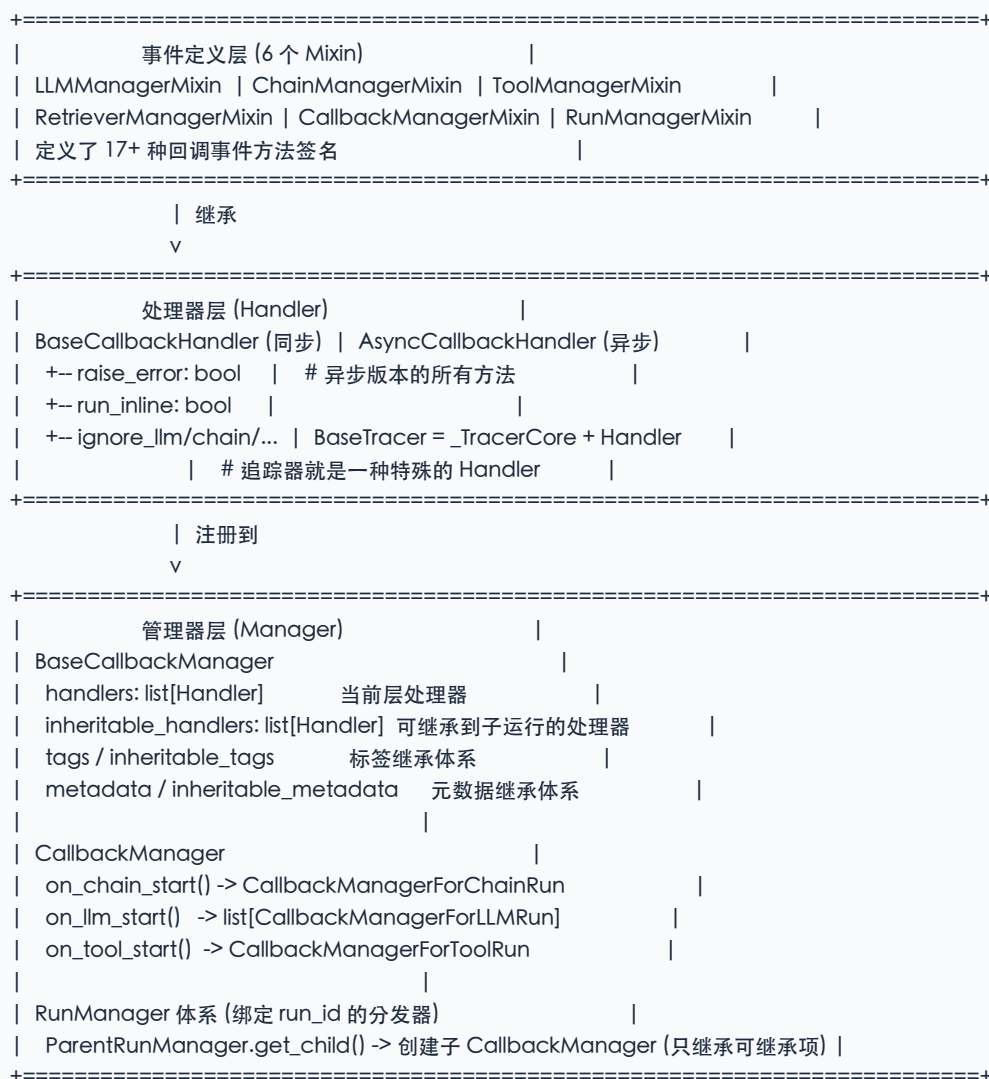
```

五、回调与追踪系统

5.1 回调系统架构图

回调系统是 LangChain 的可观测性核心，支持日志记录、追踪、事件流等功能。系统由三层构成：事件定义层（Mixin）、回调处理器层（Handler）、回调管理器层（Manager）。

图 5-1：回调系统三层架构



17 种回调事件一览：

事件方法	触发时机	忽略条件
------	------	------

on_llm_start	文本补全 LLM 开始	ignore_llm
on_chat_model_start	聊天模型开始	ignore_chat_model
on_llm_new_token	流式生成新 token	ignore_llm
on_llm_end / on_llm_error	LLM 完成 / 出错	ignore_llm
on_chain_start / end / error	Chain 开始 / 完成 / 出错	ignore_chain
on_tool_start / end / error	Tool 开始 / 完成 / 出错	ignore_agent
on_retriever_start / end / error	Retriever 开始 / 完成 / 出错	ignore_retriever
on_agent_action / finish	Agent 执行动作 / 结束	ignore_agent
on_text / on_retry	通用文本 / 重试事件	无 / ignore_retry
on_custom_event	用户自定义事件	ignore_custom_event

5.2 Chain 调用 LLM 的事件分发时序图

图 5-2: 回调事件传播完整时序

用户代码	CallbackManager	Handler_1	Handler_2(Tracer)
-- chain.invoke(input, config={"callbacks": [handler1, tracer]})			
-- CM.configure() --> 合并所有来源的回调			
	(config + 全局 + LangSmith)		
-- cm.on_chain_start(serialized, inputs, run_id=A)			
	-- handle_event -->		
	for handler: .on_chain_start()		
	.on_chain_start()		
	<-- return ChainRunManager(run_id=A)		
[链内部调用 LLM]			
-- run_mgr.get_child("llm") --> 子 CallbackManager			
(parent_run_id=A, 只继承 inheritable handlers)			
-- child_cm.on_llm_start(..., parent_run_id=A, run_id=B)			
	-- handle_event --> .on_llm_start()		
	.on_llm_start()		
	创建 Run(id=B, parent=A)		
[流式 token]			
-- llm_mgr.on_llm_new_token("Hello")			
	-- handle_event --> .on_llm_new_token()		
-- llm_mgr.on_llm_end(response)			
	-- @shielded -----> .on_llm_end()		
	(取消保护) .on_llm_end()		
	完成 Run(id=B)		
-- chain_mgr.on_chain_end(outputs)			
	-- @shielded -----> .on_chain_end()		
	.on_chain_end()		
	持久化 Run(id=A)		

5.3 追踪器与回调系统的关系

追踪器 (Tracer) 通过多重继承同时具备回调处理器和 Run 管理的能力。它被注入到 CallbackManager 的 handlers 列表中，回调管理器不区分普通 handler 和 tracer。

关键设计：BaseTracer = _TracerCore + BaseCallbackHandler

- _TracerCore: 维护 run_map (run_id -> Run), order_map (dotted_order 层级路径)
- BaseCallbackHandler: 提供所有回调事件方法的接口
- 只有根运行 (parent_run_id=None) 完成时才触发 _persist_run() 持久化
- dotted_order 格式: 'timestamp+root_id.timestamp+child_id.timestamp+grandchild_id'

六、工具系统与输出解析

6.1 工具系统核心设计

BaseTool 继承自 RunnableSerializable，因此工具本身就是 Runnable，可直接参与 LCEL 链式组合。@tool 装饰器是创建工具最便捷的方式，它根据函数签名自动推断参数 Schema。

图 6-1：工具系统类继承图

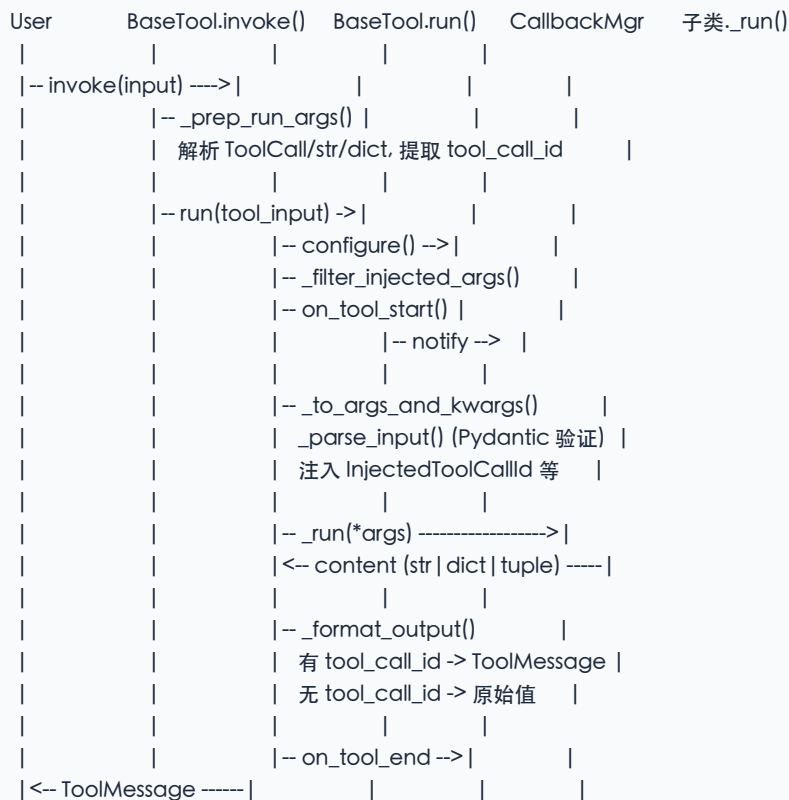
```
RunnableSerializable[str | dict | ToolCall, Any]
|
+-- BaseTool (ABC)                ← 工具基类
    字段: name, description, args_schema ← 工具标识
    return_direct: bool             ← Agent 是否直接返回
    response_format: "content" | "content_and_artifact"
    handle_tool_error: bool | str | Callable ← 错误处理策略
    方法: invoke() -> run() -> _run()    ← 模板方法模式
    |
    +-- StructuredTool              ← 结构化工具 (@tool 创建)
        | func: Callable             同步函数引用
        | coroutine: Callable         异步函数引用
        | from_function() 工厂方法
        |
    +-- Tool                        ← 简单字符串工具

@tool 装饰器:
@tool      -> StructuredTool (自动推断 schema)
@tool("name") -> StructuredTool (自定义名称)
@tool(parse_docstring=True) -> StructuredTool (从 docstring 提取描述)

依赖注入系统:
InjectedToolArg -> 标注运行时注入参数 (不暴露给 LLM schema)
InjectedToolCallId -> 自动注入 tool_call_id
```

6.2 工具调用完整时序图

图 6-2: BaseTool.invoke() -> run() -> _run() 调用链时序



错误处理: ToolException -> handle_tool_error 策略

ValidationError -> handle_validation_error 策略

其他 Exception -> 直接抛出

6.3 输出解析器体系

图 6-3: 输出解析器继承体系与解析流程

```

BaseLLMOutputParser[T] (ABC, Generic[T])    ← 最底层抽象
|
+-- BaseGenerationOutputParser[T]           ← Generation 级别解析
|
+-- BaseOutputParser[T]                     ← 文本级别解析 (主用)
    | 抽象方法: parse(text: str) -> T
    | 方法: get_format_instructions() -> str
    |
    +-- StrOutputParser                     ← 直接返回文本
    +-- ListOutputParser                    ← 解析为列表
    +-- BaseCumulativeTransformOutputParser ← 支持流式增量输出
        | +-- JsonOutputParser              ← 解析 JSON
        | +-- PydanticOutputParser          ← 解析为 Pydantic 模型
    +-- OpenAI Tools/Functions Parser       ← 解析工具调用

```

解析链路:

```

invoke(input: str | BaseMessage)
-> 包装为 Generation
-> parse_result([generation])
-> parse(result[0].text)    子类实现: 文本 -> 结构化

```

PydanticOutputParser 两层解析:

```

LLM输出文本 -> JSON解析(parse_json_markdown) -> dict -> Pydantic model_validate -> 类型安全对象

```

七、数据层：文档、检索与向量存储

7.1 数据层类图

图 7-1：数据层核心类关系图

```
Serializable
|
+-- BaseMedia
    +-- Document (page_content: str, metadata: dict, id: str | None)
    +-- Blob (data: bytes | str | None, path: PathLike, mimetype: str)
        惰性加载: from_path() 不立即读文件

BaseLoader (ABC)                ← 文档加载器
抽象方法: lazy_load() -> Iterator[Document]
便捷方法: load() -> list[Document]
          alazy_load() -> AsyncIterator[Document] (默认线程池)

VectorStore (ABC)                ← 向量存储
抽象方法: similarity_search(query, k=4) -> list[Document]
          from_texts(texts, embedding) -> VectorStore
搜索方法: similarity_search_with_score()
          similarity_search_with_relevance_scores()
          max_marginal_relevance_search()
统一入口: search(query, search_type="similarity | mmr | similarity_score_threshold")
桥接方法: as_retriever() -> VectorStoreRetriever

BaseRetriever (RunnableSerializable[str, list[Document]]) ← 检索器
抽象方法: _get_relevant_documents(query) -> list[Document]
|
+-- VectorStoreRetriever        ← 桥接 VectorStore
    vectorstore: VectorStore
    search_type: "similarity" | "mmr" | "similarity_score_threshold"
    search_kwargs: dict (k, score_threshold, filter, ...)
```

Embeddings (ABC) ← 嵌入模型
抽象方法: embed_documents(texts) -> list[list[float]]
 embed_query(text) -> list[float]

7.2 RAG 完整流程图

图 7-2: RAG (检索增强生成) 完整数据流

=== 索引阶段 (离线) ===

```

原始数据 --> BaseLoader.lazy_load() --> list[Document]
      |
      v
    TextSplitter.split_documents()
      |
      v
list[Document] (分块后)
      |
      v
Embeddings.embed_documents() --> list[list[float]]
      |
      v
VectorStore.add_documents() --> 存入向量数据库
  
```

=== 查询阶段 (在线) ===

```

用户问题: "What is LangChain?"
      |
      v
LCEL 管道: {"question": RunnablePassthrough()}
           | RunnablePassthrough.assign(
           |   context = itemgetter("question") | retriever | format_docs
           | )
           | prompt
           | model
           | StrOutputParser()
  
```

详细流程:

1. retriever._get_relevant_documents("What is LangChain?")
-> VectorStore.similarity_search(query, k=4)
-> 返回 top-k 相关文档
2. format_docs(documents) -> 拼接为上下文文本
3. prompt.format_messages(context=..., question=...)
-> [SystemMessage("基于以下上下文回答..."), HumanMessage("What is LangChain?")]
4. model._generate(messages) -> AIMessage(content="LangChain 是一个...")
5. StrOutputParser().parse(ai_message) -> "LangChain 是一个..."

八、序列化与反序列化系统

8.1 序列化机制

Serializable 是序列化的基类，通过 `is_lc_serializable()`、`get_lc_namespace()` 等方法控制序列化行为。序列化输出三种格式：`constructor`（可重建）、`secret`（敏感信息占位）、`not_implemented`（不可序列化）。

序列化类型	TypedDict	type 字段	用途
可序列化对象	SerializedConstructor	"constructor"	含 kwargs，可反序列化重建对象
敏感信息	SerializedSecret	"secret"	密钥占位符，值不被序列化
不可序列化	SerializedNotImplemented	"not_implemented"	含 repr 字符串，不可重建

`to_json()` 核心流程：检查 `is_lc_serializable()` -> 遍历模型字段过滤有意义的值 -> 收集 MRO 链上的 `lc_secrets` 和 `lc_attributes` -> 用 `_replace_secrets()` 将秘密值替换为 `SerializedSecret` -> 输出 `{"lc": 1, "type": "constructor", "id": [...], "kwargs": {...}}`

8.2 安全反序列化流程图

图 8-1: load() 安全反序列化流程

```
load(obj, allowed_objects="core", secrets_map={...})
|
+-- Reviver 类 (用作 object_hook)
|
+-- 处理 type="secret":
|   从 secrets_map 或环境变量获取实际值
|
+-- 处理 type="not_implemented":
|   抛出 NotImplementedError 或返回 None
|
+-- 处理 type="constructor":
|
+-- 1. 白名单校验 (三级安全模型):
|   "core" -> 仅 langchain_core 映射中的类
|   "all" -> 含核心 + 可信合作伙伴集成
|   显式列表 -> 仅指定的类
|
+-- 2. 命名空间校验:
|   检查 id[0] 是否在 valid_namespaces 中
|   禁止 langchain_community 通过路径加载
|
+-- 3. 动态导入:
|   importlib.import_module(namespace) -> 获取类
|
+-- 4. init_validator:
|   验证 kwargs 参数安全性
|   _block_jinja2_templates() 阻止模板注入
|
+-- 5. 实例化:
|   cls(**kwargs)

安全防护:
- _is_escaped_dict() 处理被转义的 dict (防止 'lc' key 注入)
- _block_jinja2_templates() 阻止 jinja2 模板注入攻击
- DISALLOW_LOAD_FROM_PATH 禁止特定包的路径加载
```

附录：RunnableConfig 配置体系

RunnableConfig 是贯穿所有 Runnable 组件的配置传递机制，以 TypedDict 形式定义，通过 ContextVar 在父子组件间自动传播。

字段	类型	说明
tags	list[str]	过滤标签，可继承到子运行

metadata	dict[str, Any]	元数据，JSON 可序列化，可继承
callbacks	Callbacks	回调处理器链，可继承
run_name	str	Tracer 中显示的运行名称
max_concurrency	int None	最大并发数限制
recursion_limit	int	递归调用限制，默认 25
configurable	dict[str, Any]	运行时可配置字段（如 session_id）
run_id	UUID None	唯一运行标识符

配置传播五大工具函数：

函数	功能
ensure_config(config)	补全默认值 + 合并 ContextVar 上下文配置
patch_config(config, ...)	用新值覆盖指定字段（替换 callbacks 时自动清除 run_name/run_id）
merge_configs(*configs)	合并多个配置：tags 取并集，metadata 合并，callbacks 智能合并
get_config_list(config, length)	将单个配置扩展为 batch 所需的配置列表
run_in_executor(config, func)	在线程池中运行，自动复制 contextvars.Context 到子线程